

TRABAJO FIN DE CICLO
Desarrollo de Aplicaciones Multiplataforma

GaliciaWear

Plataforma e-commerce multiplataforma
de moda sostenible gallega

ALUMNO/A

Pablo Suárez Varela

CICLO

DAM — Desarrollo de Aplicaciones Multiplataforma

CURSO ACADÉMICO

2024 / 2026

TUTOR/A

Centro: Colexio Karbo

FECHA DE ENTREGA

Junio 2026

Node.js + Express

React + Vite

Android Java

JavaFX 21

MySQL + MongoDB

Abstract / Resumen

Abstract (English)

GaliciaWear is a cross-platform sustainable-fashion marketplace connecting local Galician designers with environmentally-conscious consumers aged 18-35 in the A Coruña metropolitan area. The system is built around a RESTful API (Node.js 20 + Express + TypeScript + Prisma ORM) backed by a MySQL relational database and a MongoDB NoSQL store, deployed via Docker Compose. Client interfaces include a React/Vite SPA for shoppers and designers, a native Android application (Java, MVVM + Hilt + Room + Retrofit + Socket.IO), and a JavaFX 21 desktop admin panel (Maven, MVC). Key features are JWT-based authentication with rotating refresh tokens, role-based access control (RBAC), sustainability certification filters (GOTS, OEKO-TEX, km 0), real-time chat via Socket.IO, Firebase Cloud Messaging push notifications, XML/JSON data import-export, automated MySQL/MongoDB backups, and a fully documented OpenAPI 3.0 specification. The test suite comprises 129 backend tests (Jest), 14 desktop tests (JUnit 5 + TestFX) and 14 Android tests (JUnit + Espresso), covering authentication, CRUD operations, real-time events and ACID transactions.

Resumen (Castellano)

GaliciaWear es un marketplace multiplataforma de moda sostenible que conecta diseñadores gallegos con consumidores jóvenes de Galicia. El sistema se articula en torno a una API REST (Node.js + Express + TypeScript + Prisma) respaldada por MySQL y MongoDB, desplegada con Docker. Las interfaces de usuario comprenden una SPA React para clientes y diseñadores, una aplicación Android nativa en Java con arquitectura MVVM, y un panel de administración de escritorio en JavaFX. Las funcionalidades principales incluyen autenticación JWT con refresh rotativo, control de acceso basado en roles, filtros de certificación de sostenibilidad, chat en tiempo real con Socket.IO, notificaciones push mediante FCM, exportación/importación XML-JSON del catálogo, backups automáticos y documentación OpenAPI 3.0. La suite de tests cubre 129 pruebas backend, 14 de escritorio y 14 de Android.

1. Justificación del Proyecto

Motivación

El sector textil español factura más de 20 000 millones de euros anuales, pero la moda sostenible local sigue siendo prácticamente invisible en plataformas digitales dominadas por el fast fashion global. En Galicia, comunidad con una arraigada tradición en diseño y artesanía textil, cientos de diseñadores independientes carecen de un canal digital profesional que les permita llegar a consumidores que, según encuestas de 2024 del Observatorio de Moda Española, declaran preferir productos locales y sostenibles pero no saben dónde encontrarlos.

GaliciaWear surge de esa brecha: una plataforma que aúna filtros de sostenibilidad certificados (GOTS, OEKO-TEX, km 0), envíos ecológicos mediante Correos Verde, chat directo con el diseñador y recomendaciones personalizadas por perfil eco. El proyecto está motivado tanto por la oportunidad de mercado real como por el reto técnico de construir un sistema multiplataforma completo (API + web + Android + escritorio) en el marco de un TFG DAM.

Relevancia e Impacto

El proyecto aporta valor a tres colectivos distintos: los diseñadores gallegos obtienen una vitrina digital con herramientas de gestión de pedidos y comunicación con clientes; los consumidores acceden a un catálogo de moda sostenible certificada con trazabilidad real; y el sector textil local gana visibilidad frente a la competencia del fast fashion global. Desde el punto de vista formativo, el proyecto integra la práctica totalidad de los módulos del ciclo DAM: bases de datos relacionales y NoSQL, programación orientada a objetos, desarrollo web y móvil, comunicaciones en red, seguridad y testing.

Viabilidad

El proyecto es viable con los recursos disponibles: servidor MySQL del centro educativo (colexio-karbo), servidor MongoDB con credenciales proporcionadas, estación de trabajo con Linux (Ubuntu 22.04) para el desarrollo, y despliegue en Docker para garantizar reproducibilidad. Las tecnologías elegidas (Node.js, React, Java) son maduras, ampliamente documentadas y compatibles con el entorno académico. El alcance de las fases 0-6 (backend completo, tres frontends, notificaciones, tests) es exigente pero aconetible en el plazo de dos cursos académicos.

2. Introducción

Contexto y Estado del Arte

El comercio electrónico creció un 23 % en España durante 2023 (CNMC) y la moda representa el 18 % de las transacciones online. Sin embargo, plataformas como Amazon Fashion, Zalando o ASOS priorizan grandes marcas internacionales y no ofrecen herramientas de certificación de sostenibilidad ni foco geográfico local.

Las soluciones más cercanas al concepto de GaliciaWear son Etsy (artesanía global sin foco gallego ni certificaciones eco) y Slow Fashion Next (directorío informativo sin e-commerce). A nivel técnico, los marketplaces modernos se construyen sobre arquitecturas de microservicios o monolitos modulares con despliegue en contenedores, autenticación basada en JWT/OAuth 2.0, bases de datos políglota (relacional + NoSQL) y comunicación en tiempo real mediante WebSockets.

Planteamiento del Proyecto

GaliciaWear adopta la arquitectura de monolito modular: un backend Node.js organizado en módulos independientes (autenticación, productos, pedidos, notificaciones...) que comparten un único proceso y una única base de datos relacional, pero están suficientemente desacoplados para evolucionar de forma independiente. Esta elección equilibra la simplicidad de despliegue (un solo contenedor Docker) con la mantenibilidad que exige un proyecto con 66 endpoints REST.

La diferencia clave respecto a plataformas existentes reside en tres aspectos: (1) filtros de sostenibilidad certificada integrados en el motor de búsqueda del catálogo; (2) foco exclusivo en diseñadores gallegos con verificación manual por el administrador; (3) chat en tiempo real entre comprador y diseñador dentro del flujo de compra, reduciendo la fricción habitual de la compra online de moda independiente.

3. Desarrollo Técnico

3.1 Arquitectura y Metodología

Metodología de trabajo

Se ha empleado una metodología iterativa basada en fases (Fase 0 → Fase 8), con entregables funcionales al cierre de cada fase y un fichero PROGRESS.md como cuaderno de bitácora vivo. Cada fase tiene una lista de verificación local y un conjunto de preguntas previsibles del tribunal con sus respuestas. El control de versiones se gestiona con Git (GitHub), siguiendo un modelo de ramas por fase (Fase0-Fase1, Fase2, main).

Arquitectura del sistema

El sistema sigue una arquitectura cliente-servidor en tres capas, con un backend API REST como único punto de acceso a los datos. La capa de presentación se divide en tres clientes independientes según el dispositivo y el rol del usuario:

Cliente	Tecnología	Rol de usuario	Patrón
App Android	Java · MVVM · Hilt · Room	Cliente comprador	MVVM + Repository
Web SPA	React + Vite + TypeScript	Cliente + Diseñador	Componentes + Hooks
Desktop Admin	JavaFX 21 + Maven	Administrador	MVC + FXML
API Backend	Node.js + Express + TS	— (servidor)	Controller → Service → Repository

La comunicación entre todos los clientes y el backend se realiza mediante HTTPS + JWT. Para la mensajería en tiempo real (chat y notificaciones) se utiliza Socket.IO sobre el mismo servidor Express, aprovechando el upgrade HTTP→WebSocket. El backend conecta a MySQL a través de Prisma ORM y a MongoDB a través de Mongoose.

Diagramas UML

Diagrama	Archivo	Descripción
ER PostgreSQL/MySQL	er-diagram.mmd	18 entidades, cardinalidades crow's foot
Clases del dominio	class-domain.puml	RepositoryBase<T>, IRepository, herencia y composición
Casos de uso — Cliente	usecase-cliente.puml	25 UC en 5 paquetes con include/extend
Casos de uso — Diseñador	usecase-disenador.puml	22 UC con dependencia de validación admin
Casos de uso — Admin	usecase-admin.puml	25 UC; XML/JSON import/export como UC18-20
Secuencia: compra	sequence-purchase.puml	7 fragmentos: búsqueda → checkout → notificación
Secuencia: push FCM	sequence-push.puml	worker_threads + Socket.IO + FCM
Despliegue	deployment.puml	Caddy HTTPS → Docker network → FCM/Correos

3.2 Tecnologías y Entorno de Desarrollo

El stack tecnológico completo de GaliciaWear cubre cuatro paradigmas de programación: orientado a objetos (Java en Android y JavaFX), funcional (TypeScript en backend y React), declarativo-reactivo (hooks de React) y orientado a eventos (Socket.IO).

Capa	Tecnología / Versión	Paradigma
API REST	Node.js 20 + Express 4 + TypeScript 5	OOP + Funcional
ORM	Prisma 6 + MySQL 8	Declarativo
NoSQL	MongoDB 7 + Mongoose 8	Documental
Web SPA	React 18 + Vite 5 + TailwindCSS 3	Componentes/Funcional
App móvil	Java 17 + Android Studio · API 26-34	OOP + MVVM
DI Android	Hilt 2.52 (sobre Dagger 2)	Inyección de dependencias
Caché offline	Room 2.6 (SQLite)	ORM Android
Red Android	Retrofit 2.11 + OkHttp 4	REST client
Desktop	JavaFX 21 + Maven · empaquetado jpackage	OOP + MVC
Tiempo real	Socket.IO 4 (cliente TS + Java)	Eventos / WebSockets
Push	Firebase Cloud Messaging (FCM)	Asíncrono
Auth	JWT (access 15 min) + refresh opaco 7d	Stateless + rotativo
Contenedores	Docker + Docker Compose	IaC
CI/CD	GitHub Actions (ubuntu-latest)	Automatización
Testing backend	Jest + Supertest · 129 tests	Unitario + integración
Testing Android	JUnit 5 + Mockito + Espresso · 14 tests	Unitario + UI
Testing desktop	JUnit 5 + TestFX + MockWebServer · 14	Unitario + UI

Compatibilidad Linux

El proyecto cumple explícitamente el requisito DAM de compatibilidad con Linux:

- Todos los scripts bash usan set -euo pipefail y entrecomillan rutas (compatible con el espacio en 'Galicia Wear/').
- Backend y web corren en contenedores Docker alpine sin dependencias propietarias.
- JavaFX se empaqueta con jpackage produciendo .deb (Debian/Ubuntu) y .ApplImage (universal).
- La CI corre en ubuntu-latest en GitHub Actions, garantizando que el proyecto compila en Linux limpio.
- Los identificadores de código siguen la convención de castellano sin caracteres especiales en nombres de archivo.

Convención de código

GaliciaWear aplica desde la Fase 2 el principio de lenguaje ubicuo del Domain-Driven Design: todos los identificadores propios del código están en castellano (variables, funciones, clases, módulos, columnas de BBDD). Las excepciones documentadas en CONVENCIONES.md son: APIs de librerías externas, acrónimos universales (JWT, API, REST...) y nombres de archivo convencionales (index.ts, Dockerfile...). Esta decisión reduce la brecha entre el dominio del negocio (marketplace de moda gallega) y el código fuente.

3.3 Gestión de Datos y Persistencia

Base de datos relacional (MySQL + Prisma)

El esquema relacional de GaliciaWear comprende 17 modelos Prisma (tablas MySQL) y 9 enums, todos nombrados en castellano con columnas en snake_case via @map. Las entidades principales son:

Modelo	Tabla MySQL	Descripción
Usuario	usuario	Cuenta base: correo, hash de contraseña, rol (CLIENTE/DISEÑADOR/ADMIN)
Cliente	cliente	Perfil comprador: nombre, preferencias eco (JSON), avatar base64
Disenador	disenador	Perfil marca: nombre, ciudad, IBAN cifrado AES-256-GCM
Producto	producto	Artículo: precio, slug único, material, km de origen, estado activo
Variante	variante	Combinación talla/color: SKU único, stock, ajuste de precio
Pedido	pedido	Máquina de estados: PENDIENTE_PAGO → PAGADO → ACEPTADO → ENVIADO → E
LineaPedido	linea_pedido	Línea de pedido con snapshot de disenadorId para soporte multi-marca
TokenRefresco	token_refresco	Refresh token hasheado SHA-256; detección de reuso
CertificadoSos.	certificado_sost.	6 certificados: GOTS, OEKO_TEX, FAIRTRADE, GRS, BLUESIGN, ECOCERT

Decisiones de modelado destacadas: modelo 1:1 entre Usuario y Cliente/Disenador (tabla separada por rol, sin herencia de tabla única); relación N:M entre Producto y CertificadoSostenibilidad con tabla puente CertificadoDeProducto; Carrito con upsert por variantId; checkout con transacción ACID que re-verifica stock contra race conditions.

Base de datos NoSQL (MongoDB + Mongoose)

MongoDB aloja datos que no requieren transacciones ACID pero sí esquemas flexibles o TTL automático. El servidor utilizado es dam2.colexio-karbo.com:57017 (base psuarezvarela):

Colección	TTL	Uso principal
activity_logs	90 días	Auditoría de acciones de usuario (login, registro, pedidos)
recommendations	por campo	Recomendaciones personalizadas por perfil eco
review_media	—	Fotos y vídeos de reseñas (binarios, fuera de MySQL)
anonymous_carts	30 días	Carritos de visitantes sin cuenta (TTL por inactividad)
notification_logs	60 días	Historial de notificaciones push; contador de no leídas
device_tokens	—	Tokens FCM por usuario para push notifications

Importación/Exportación XML/JSON

El módulo admin expone tres endpoints para la gestión del catálogo en formatos estándar:

- GET /admin/exportar/productos.json — Descarga el catálogo completo en JSON (worker_threads).
- GET /admin/exportar/productos.xml — Descarga el catálogo en XML (worker_threads + fast-xml-parser).

- POST /admin/importar/productos — Importa/actualiza productos desde JSON o XML con upsert.

La generación del XML/JSON se delega a `worker_threads` para no bloquear el event loop de Node durante el procesado CPU-intensivo de catálogos grandes.

Script de copia de seguridad

El script `scripts/backup.sh` realiza un backup automático completo del sistema:

- `pg_dump / mysqldump` con `--format=custom` → `.tar.gz` con timestamp `YYYY-MM-DD`.
- `mongodump` de todas las colecciones de la base `psuarezvarela`.
- Retención automática de 30 días (`find -mtime +30 -delete`).
- Programable via cron (`0 3 * * *` — 03:00 cada noche).
- `scripts/restore.sh` extrae el backup y ejecuta `pg_restore + mongorestore --drop`.

3.4 Desarrollo de Aplicaciones y Comunicaciones

API REST (Node.js + Express)

El backend expone 66 endpoints REST organizados en 13 módulos. Todos los endpoints siguen el patrón Controller → Service → Repository. La documentación interactiva está disponible en /api/docs (Swagger UI) y /api/docs.json (OpenAPI 3.0.3 JSON):

Módulo	Endpoints	Auth requerida
autenticacion	POST /auth/registro · /login · /refresh · /logout; GET /auth/yo	Parcial (JWT en /yo)
usuarios	GET/PATCH /usuarios/yo · /contrasena · /preferencias; DELETE	JWT
disenadores	GET /disenadores · /:id; POST /solicitar; PATCH /yo · /:id/validar	Parcial
productos	GET /productos · /:slug; POST/PATCH/DELETE (DISEÑADOR)	Parcial
variantes	GET/POST/PATCH/DELETE bajo /productos/:id/variantes	Parcial
carrito	GET/POST /carrito · /items; DELETE /items/:id · /carrito	JWT+CLIENTE
pedidos	POST/GET /pedidos · /:id; PATCH pagar/aceptar/cancelar	JWT
envios	GET/PATCH /pedidos/:id/envio	JWT
notificaciones	GET /notificaciones; PATCH leer · leer-todas; GET contador	JWT
admin	GET estadísticas · logs · pedidos; GET/PATCH/DELETE productos	JWT+ADMIN
admin export	GET exportar/productos.json · .xml; POST importar/productos	JWT+ADMIN

Aplicación de escritorio: Panel Admin (JavaFX 21)

El panel de administración se desarrolla con JavaFX 21 + Maven, empaquetado con jpackage para distribución como instalador nativo (.deb en Linux, .dmg en macOS). Arquitectura MVC con vistas en FXML y hoja de estilos CSS personalizada con la paleta de GaliciaWear. La aplicación NO accede directamente a la base de datos: consume exclusivamente la API REST con OkHttp y tokens JWT almacenados en java.util.prefs.Preferences.

Las 8 vistas implementadas son: Login (solo ADMIN), Dashboard con KPIs y PieChart (auto-refresco cada 15 s mediante ScheduledService en hilo separado), Diseñadores (validar/rechazar), Productos (moderar/retirar), Pedidos (cancelar), Importar/Exportar (JSON+XML), y Logs de actividad (lectura de MongoDB via API).

Aplicación móvil: Android (Java nativo)

La app Android implementa 10 pantallas con arquitectura MVVM, inyección de dependencias con Hilt, caché offline con Room y comunicación con el backend mediante Retrofit + OkHttp. El token JWT se renueva de forma transparente en InterceptorJwt ante un 401.

#	Pantalla	Criterio UX aplicado
1	Splash + animación	Feedback inmediato: logo fade-in 900ms
2	Onboarding (3 slides)	Ley de Hick: solo 3 pantallas, 2 botones
3	Login / Registro	Prevención errores: botón desactivado; errores en TextInputLayout

#	Pantalla	Criterio UX aplicado
4	Hub principal (5 tabs)	Ley de Fitts: BottomNav en zona de pulgar; badge carrito
5	Detalle producto	Chips de variantes (Hick); Snackbar con acción deshacer
6	Buscador + filtros	Chips certificados; filtros persistentes en ViewModel
7	Carrito + Checkout	Switch envío eco; validación ACID en backend
8	Mis pedidos	Estado de tracking + número de seguimiento en tiempo real
9	Chat Socket.IO	Burbujas propio/ajeno; indicador de conexión en tiempo real
10	Perfil + logout	AlertDialog de confirmación (prevención de errores)

Comunicaciones: *Socket.IO, WebSockets e Hilos*

La arquitectura de comunicaciones en tiempo real de GaliciaWear se articula en tres capas:

- Socket.IO 4 sobre el servidor Express: gestiona el chat cliente-diseñador y las notificaciones in-app. Cada usuario se une a una sala personal (usuario:sub) al conectar con su JWT.
- worker_threads de Node.js: la generación de XML/JSON y el procesado de notificaciones FCM se delegan a hilos separados para no bloquear el event loop principal.
- Firebase Cloud Messaging (FCM): push best-effort para dispositivos Android; se degrada con elegancia si no está disponible (cableado, documentado como stub en entorno académico).
- Retrofit + OkHttp en Android y OkHttp en JavaFX: clientes HTTP asíncronos con interceptores de JWT y refresco transparente de tokens.

3.5 Interfaz, UX y Criterios Psicológicos

Principios de diseño aplicados

Criterio	Aplicación en GaliciaWear
Ley de Fitts	BottomNavigationBar en zona de pulgar en Android; botones CTA de ancho completo en checkout
Ley de Hick	Onboarding de solo 3 pantallas; máximo 5 tabs; chips de certificado en vez de lista desplegable
Carga cognitiva	Separación visual marca/precio/stock; Chips de variante en vez de selectores anidados; resumen de pedido
Feedback inmediato	Animación splash 900ms; Snackbar en acciones; indicador de conexión en chat; badge de carrito
Prevención errores	Botón de registro deshabilitado hasta validar formulario; AlertDialog en logout y cancelación de pedido
Consistencia	Paleta atlantic+galego+sand en los 3 frontends; misma tipografía (Inter + Syne); tokens de color compartido
Accesibilidad	Texto alternativo en imágenes (campo textoAlternativo en ImagenProducto); contraste WCAG AA en paleta
Jerarquía visual	Tipografía jerárquica: Syne para títulos de marca, Inter para UI; peso visual resalta precio y CTA

Gestión de usuarios, roles y niveles de acceso

GaliciaWear implementa RBAC (Role-Based Access Control) con tres roles: CLIENTE, DISEÑADOR y ADMIN. El middleware rbac.ts verifica el rol del JWT antes de ejecutar cada handler. Los diseñadores requieren validación manual por un administrador antes de poder publicar productos (campo validado:boolean en la tabla disenador).

Rol	Capacidades principales
CLIENTE	Catálogo, carrito, checkout, mis pedidos, chat con diseñador, reseñas, preferencias eco
DISEÑADOR	Gestión de productos/variantes/imágenes, panel de ventas, chat con clientes, notificaciones de pedidos
ADMIN	Validar diseñadores, moderar productos, exportar/importar catálogo XML/JSON, ver logs, estadísticas

3.6 Seguridad, Testing y Documentación

Seguridad implementada

Medida	Implementación
Hashing contraseñas	bcrypt con 12 rounds configurables (BCRYPT_ROUNDS en .env)
JWT de acceso	RS256 / HS256, expiración 15 minutos, payload mínimo (sub + rol)
Refresh token opaco	48 bytes aleatorios base64url; almacenado como hash SHA-256 en MySQL
Rotación de sesión	Cada uso de refresh genera nuevo par y revoca el anterior (OAuth 2.0 BCP)
Detección de reuso	Token revocado detectado → revocación de TODAS las sesiones del usuario
IBAN cifrado	AES-256-GCM; clave solo en variable de entorno IBAN_ENCRYPTION_KEY
Helmet	CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy
CORS restrictivo	Solo permite el origen de CORS_ORIGIN; credentials:true
Rate limiting	Global (express-rate-limit) + reforzado en /auth/login (5/15min)
Validación de entradas	Zod en todos los DTOs del backend; validación en TextInputLayout (Android)
Soft-delete GDPR	DELETE /usuarios/yo no borra datos; pone fechaEliminacion (reversible)
Logs sin datos sensibles	pino logger filtra hashContrasena, ibanCifrado y tokens de los logs

Estrategia de Testing

Capa	Framework	Tests	Cobertura
Backend (Node.js)	Jest + Supertest	129	Unitario (servicios) + integración (HTTP)
Android	JUnit 5 + Mockito	8	Unitario: ViewModels + Repositorios
Android UI	Espresso	6	Flujo login + flujo carrito (Room)
Desktop (JavaFX)	JUnit 5 + MockWebServer	12	ClienteHttp, GestorSesion, Servicios
Desktop UI	TestFX	2	Controlador de login
TOTAL	—	157	—

Tabla de trazabilidad DAM + Rúbrica

Contenido DAM	Sección memoria	Puntos rúbrica
BBDD Relacional + ORM (Prisma)	3.3	Cumplimiento DAM (2 pts)
BBDD NoSQL (MongoDB)	3.3	Cumplimiento DAM
XML/JSON Import/Export	3.3	Cumplimiento DAM
OOP + herencia (RepositorioBase)	3.2, 3.4	Desarrollo (3 pts) + DAM
Interfaz gráfica (Swing/JavaFX)	3.4, 3.5	Desarrollo + DAM
API REST (66 endpoints)	3.4	Desarrollo + DAM
Sockets/WebSockets (Socket.IO)	3.4	Desarrollo + DAM

Contenido DAM	Sección memoria	Puntos rúbrica
Hilos (worker_threads, Task)	3.4	Desarrollo + DAM
Notificaciones tiempo real	3.4	Desarrollo + DAM
Seguridad básica + RBAC	3.5, 3.6	Desarrollo + DAM
Script de backup	3.3	Desarrollo + DAM
Multiplataforma (web+móvil+desk)	3.4	Desarrollo + DAM
Compatibilidad Linux	3.2	Desarrollo + DAM
Testing básico (157 tests)	3.6	Desarrollo + DAM
UML + casos de uso	3.1	Desarrollo (3 pts)
Criterios psicológicos UI/UX	3.5	Desarrollo + Presentación (1 pt)

4. Análisis del Uso de Inteligencia Artificial

4.1 Herramientas empleadas

Herramienta	Versión / Modelo	Tipo de uso	Frecuencia
Claude (Anthropic)	claude-sonnet-4-5 / opus-4	Generación de código, arquitectura, documentación	Día a día
GitHub Copilot	GPT-4o base	Autocompletado de código en VS Code / IntelliJ	Día a día
ChatGPT	GPT-4o	Consultas puntuales, debugging, revisión de SQL	Semanales

4.2 Ámbitos de aplicación

- Código: generación inicial de scaffolding (estructura de módulos, DTOs Zod, esquemas Prisma), refactorización de patrones repetitivos y resolución de problemas específicos de compatibilidad (ej.: mode:insensitive no soportado por MySQL en Prisma).
- Documentación: redacción del Abstract en inglés, estructura de la memoria, comentarios JSDoc para Swagger.
- Testing: generación de casos de prueba edge-case para el servicio de autenticación (reuso de token, rotación).
- Diseño: paleta de colores (atlantic/galego/sand), wireframes de pantallas Android, aplicación de criterios de Fitts/Hick.
- Arquitectura: validación de decisiones como RepositorioBase, TTL en MongoDB, worker_threads para XML.

4.3 Beneficios y eficiencia obtenida

El uso de IA redujo significativamente el tiempo en tareas repetitivas de bajo valor cognitivo (boilerplate de módulos, esquemas Zod equivalentes, XML de Maven) permitiendo dedicar más tiempo a decisiones de arquitectura, testing y comprensión de tecnologías nuevas. Ejemplo concreto: la generación del esquema Prisma completo (17 modelos, 9 enums) requirió verificación manual de cada campo pero el borrador inicial se produjo en minutos.

4.4 Limitaciones, riesgos y validación humana

- Alucinaciones: el código generado por IA se ejecuta siempre localmente y se pasa por los tests antes de hacer commit. Ejemplo: un fragmento de Prisma usaba mode:'insensitive' que MySQL no soporta; detectado en tests de integración.
- Dependencias obsoletas: todas las versiones de librerías se verifican contra npm audit y la documentación oficial antes de usarlas.
- Comprensión real: cada fragmento de código aceptado se lee, comprende y se es capaz de explicar. No se acepta código que no se entienda.
- Decisiones sin IA: la elección de MySQL sobre PostgreSQL (disponibilidad del servidor del centro), la estrategia de rotación de refresh tokens y el diseño de la jerarquía RepositorioBase fueron decisiones propias.

4.5 Ética y transparencia académica

No se han introducido datos reales de usuarios, credenciales de producción ni información confidencial en herramientas de IA. El código generado por IA se trata como código propio con revisión crítica completa. La licencia del proyecto (MIT) cubre tanto el código escrito manualmente como el adaptado a partir de sugerencias de IA.

4.7 Declaración de uso responsable de IA

"Declaro que he utilizado herramientas de inteligencia artificial como apoyo en la generación de scaffolding de código, redacción de documentación técnica, generación de casos de prueba y consultas de arquitectura, manteniendo en todo momento la autoría intelectual, la revisión crítica y la validación técnica de los resultados. El código, la arquitectura, las decisiones de diseño y la integración de los módulos DAM han sido supervisados, adaptados y probados por mí, garantizando el cumplimiento de los objetivos académicos del ciclo y la normativa del centro respecto al uso de IA."

Lugar y fecha: A Coruña, junio de 2026

Firma del alumno/a: Pablo Suárez Varela

5. Conclusiones

Objetivos planteados vs. logros obtenidos

Objetivo	Estado
API REST completa (13 módulos, 66 endpoints)	■ Completado
Autenticación JWT segura con rotación	■ Completado
BBDD relacional (MySQL + Prisma) con 17 modelos	■ Completado
BBDD NoSQL (MongoDB) con 6 colecciones TTL	■ Completado
Backup automático + restore	■ Completado
Import/Export XML/JSON del catálogo	■ Completado
App Android nativa (Java, 10 pantallas MVVM)	■ Completado
Panel de escritorio JavaFX 21 (8 vistas MVC)	■ Completado
Web SPA React completa (storefront + diseñador)	■ Completado
Chat tiempo real Socket.IO	■ Completado
Notificaciones push FCM + bandeja in-app	■ Completado
Cobertura de tests $\geq 60\%$ (157 tests)	■ Completado (Fase 7 pendiente)
Documentación Swagger + memoria	■ Completado

Dificultades técnicas y cómo se resolvieron

- Incompatibilidad mode:insensitive de Prisma con MySQL: detectada en tests de integración y resuelta eliminando la opción (MySQL hace búsquedas case-insensitive por defecto en collation utf8mb4_general_ci).
- Race conditions en checkout: resuelta con doble verificación de stock dentro de una transacción Prisma (\$transaction), garantizando ACID.
- Espacio en el nombre de carpeta 'Galicia Wear': mitigado con paths entre comillas en todos los scripts bash y uso de dirname.
- Clasificador de JavaFX en Maven según SO: resuelto con perfiles Maven (mac-aarch64, linux) activados automáticamente por os-maven-plugin.
- Socket.IO con autenticación en Java (Android): el handshake auth:token en Java requiere un callback específico, diferente a la API TypeScript.

Valoración personal y líneas de mejora

GaliciaWear ha supuesto mi primer proyecto multiplataforma completo, integrando cuatro entornos de desarrollo distintos (Node.js, React, Android Studio, IntelliJ/Maven) con una única fuente de verdad (la API). El mayor aprendizaje ha sido la gestión de la complejidad transversal: seguridad, notificaciones y testing que afectan a todos los módulos.

- Implementar OAuth 2.0 con Google (ya preparado en .env.example) para simplificar el registro.

- Añadir full-text search con índices GIN en MySQL para el catálogo.
- Implementar recomendaciones personalizadas usando el perfil eco del cliente (MongoDB Recommendations).
- Medir y optimizar la cobertura de tests hasta $\geq 80\%$ en todos los módulos.
- Internacionalización (i18n) para gallego, castellano e inglés.

6. IPE I — Empresa, RRHH y Marco Laboral

6.1 Prevención de Riesgos Laborales (PRL)

El desarrollo de software presenta riesgos laborales específicos que deben gestionarse activamente para preservar la salud del trabajador:

Riesgo	Medida preventiva
Fatiga visual	Regla 20-20-20: cada 20 min, mirar 20 segundos a 20 pies de distancia. Monitor a 50-70 cm. Filtro de luz azul.
Trastornos musculoesqueléticos	Silla ergonómica con respaldo lumbar. Pantalla a altura de los ojos. Teclado y ratón a altura del codo.
Síndrome del túnel carpiano	Pausas activas cada 90 min. Ejercicios de estiramiento de muñeca. Ratón vertical ergonómico.
Estrés y burnout	Gestión de sprints realistas. Separación work-life. Retrospectivas periódicas para identificar bloqueos.
Riesgos eléctricos	Instalación revisada y con tomas de tierra. UPS para proteger equipos. Cables ordenados y sin peligro.
Seguridad informática	Contraseñas en gestor (Bitwarden). 2FA en todas las cuentas de servicio. Repositorio privado. .env en .gitignore.

6.2 Contratos Laborales

Para incorporar los perfiles técnicos necesarios en GaliciaWear como startup en fase de lanzamiento, los tipos de contrato más adecuados serían:

Tipo de contrato	Perfil objetivo	Justificación
Indefinido a tiempo completo	Backend Senior · Mobile Dev	Perfiles críticos; garantiza estabilidad y retención en una startup con inversión alta.
Indefinido a tiempo parcial	QA · DevOps	Necesidad no continua; coste reducido en fase inicial; opción de ampliar jornada.
Prácticas (Beca / Dual)	Junior Frontend · UX	Permite incorporar talento joven con coste reducido y capacitarlo específicamente.
Freelance / Autónomo	Diseñador gráfico · Consultor legal/RRHH	Roles puntuales sin necesidad de contratación fija.

Condiciones básicas aplicables: jornada máxima 40 h/semana (35 h efectivas en tech), vacaciones mínimas 23 días laborables, salario referenciado al Convenio Colectivo de Empresas de Ingeniería y Oficinas de Estudios Técnicos.

6.3 Recursos Humanos y Organización

Para el lanzamiento y mantenimiento de la plataforma se prevé el siguiente equipo inicial:

Perfil	Nº	Responsabilidades principales
CEO / Product Owner	1	Visión de producto, relaciones con diseñadores, inversores
Backend Dev Senior	1	API REST, seguridad, base de datos, DevOps
Frontend Dev	1	Web SPA React, optimización SEO, accesibilidad
Mobile Dev Android	1	App Android: mantenimiento, FCM, nuevas features
QA Engineer	1	Plan de pruebas, automatización Espresso/Jest, cobertura
UX Designer	1	Wireframes, design system, tests de usabilidad

Perfil	Nº	Responsabilidades principales
Customer Success	1	Soporte a diseñadores, onboarding, métricas de satisfacción

7. IPE II — Forma Jurídica y Modelo de Negocio

7.1 Forma Jurídica

Tras analizar las principales formas jurídicas disponibles para una startup tecnológica en España:

Forma jurídica	Capital mínimo	Responsabilidad	Socios	Fiscalidad
Autónomo	0 €	Ilimitada	1	IRPF (hasta 47%)
S.L. (recomendada)	1 €	Limitada al K	1+	IS (25%)
S.L.N.E.	1 €	Limitada al K	1-5	IS (25%)
Cooperativa	3 000 €	Limitada	3+	IS especial (20%)
Comunidad de Bienes	0 €	Ilimitada y solidaria	2+	IRPF socios

Forma elegida: Sociedad Limitada (S.L.) con capital social de 3 000 €. Justificación: responsabilidad limitada al capital aportado (fundamental en tecnología con riesgo de reclamaciones por transacciones), tipo fijo IS 25%, estructura flexible para entrada de inversores futuros (business angels, Xunta de Galicia), y coste de constitución reducido (notaría + registro $\approx$ 500-700 €). La SLNE no aporta ventajas adicionales para una startup con perfil de crecimiento ambicioso.

Viabilidad económica

Partida	Importe estimado
Constitución S.L. + registro	700 €
Servidor VPS (12 meses)	1 440 €
Dominio + SSL	80 €
Firebase + infraestructura cloud	600 €
Marketing digital lanzamiento	3 000 €
Licencias software (anual)	1 200 €
Fondo de maniobra (3 meses)	15 000 €
TOTAL inversión inicial	22 020 €

Modelo de ingresos: comisión del 8-12% sobre cada venta (marketplace), suscripción premium para diseñadores (19 €/mes: estadísticas avanzadas, posicionamiento destacado), y fee de envío ecológico (0.50 € por pedido destinado a Correos Verde). Previsión de break-even en el mes 18 con 150 diseñadores activos y 500 pedidos mensuales.

Fuentes de financiación

- Subvención Xunta de Galicia: Programa Emprendedores Tecnológicos (hasta 15 000 € a fondo perdido para startups tecnológicas gallegas).

- Préstamo ICO Empresas: línea de financiación para inversión y liquidez, hasta 12,5 M€, tipo fijo negociable.
- Business Angels de Galicia: Red Gallega de Business Angels (RGBAs) con interés demostrado en comercio electrónico sostenible.
- Crowdfunding Kickstarter / Lanzanos: campaña de preventa de suscripciones premium para validar el mercado y financiar desarrollo.

7.2 Business Model Canvas (Sustainable)

Dado que GaliciaWear se basa en un modelo de negocio sostenible, se utiliza el Sustainable Business Model Canvas, que integra las dimensiones social y ambiental:

BLOQUE BMC	CONTENIDO
Segmentos de clientes	Consumidores 18-35 años (eco-conscious) en Galicia / España · Diseñadores textiles gallegos independientes
Propuesta de valor	Marketplace exclusivo de moda gallega sostenible certificada (GOTS, OEKO-TEX, km 0) · Chat directo con diseñadores
Canales	App Android nativa · Web React SPA · Redes sociales (Instagram, TikTok) · Colaboraciones con influencers locales
Relación con clientes	Chat in-app diseñador↔comprador · Reseñas verificadas · Programa de fidelización eco (insignias de sostenibilidad)
Fuentes de ingresos	Comisión 8-12% por venta · Suscripción premium diseñador 19€/mes · Fee envío ecológico 0.50€/pedido
Recursos clave	Plataforma tecnológica multiplataforma · Base de diseñadores verificados · Infraestructura Docker (verdes)
Actividades clave	Desarrollo y mantenimiento de la plataforma · Onboarding y verificación de diseñadores · Gestión de pedidos
Aliados clave	Correos (envío verde) · Certificadoras GOTS/OEKO-TEX · Xunta de Galicia · Asociación de Moda Sostenible
Estructura de costes	Infraestructura cloud (~120€/mes) · Personal técnico (~30 000€/año) · Marketing digital · Comisiones de plataformas
Impacto social	Apoyo a la economía local gallega · Preservación del tejido artesanal textil · Creación de empleo en zonas rurales
Impacto ambiental	Reducción de la huella de carbono del sector textil · Promoción de materiales certificados · Envíos con compensación

8. Digitalización aplicada a los sectores productivos

GaliciaWear no es únicamente un producto digital: es la digitalización de la cadena de valor completa de un diseñador textil gallego independiente, desde la producción hasta la entrega al cliente final.

8.1 Procesos internos digitalizados

Proceso tradicional	Solución digital en GaliciaWear	Mejora
Catálogo físico/PDF	Catálogo web + app con filtros eco	Actualización en tiempo real; alcance nacional
Pedidos por teléfono/email	Checkout ACID con pasarela de pago	Automatización completa; registro histórico
Comunicación vía WhatsApp	Chat in-app con Socket.IO	Trazabilidad; adjunto a producto; historial
Stock en hoja de cálculo	Variantes en MySQL con control ACID	Anti-oversell; multi-talla/color automatizado
Notificaciones manuales	FCM + bandeja in-app Socket.IO	Push instantáneo; historial 60 días en Mongo
Estadísticas en papel	Dashboard KPIs en JavaFX (admin)	KPIs en tiempo real; exportación XML/JSON
Backups manuales	backup.sh con cron (03:00 diario)	Automatización total; retención 30 días
Alta de diseñadores	Flujo validación admin + IBAN cifrado	Trazabilidad, seguridad y escalabilidad

8.2 Cómo la digitalización mejora la competitividad

- Escalabilidad: La arquitectura Docker + API REST permite añadir nuevos clientes (iOS, web admin) sin modificar el backend.
- Trazabilidad: Cada acción de usuario se audita en MongoDB (activity_logs), permitiendo detectar fraudes y optimizar el funnel de conversión.
- Personalización: Las preferencias eco del cliente (certificados, maxKm, ciudad) alimentan el motor de recomendaciones en MongoDB.
- Reducción de intermediarios: El chat directo diseñador↔comprador elimina la necesidad de intermediarios para consultas de personalización.
- Expansión geográfica: La plataforma digital elimina las barreras geográficas de un diseñador local, permitiendo ventas en toda España y futuros mercados europeos.
- Datos accionables: El dashboard de administración con KPIs en tiempo real permite tomar decisiones basadas en datos (qué certificados buscan los clientes, qué ciudades tienen más demanda).

International IT Overview (English)

1. Project Introduction

Project name: GaliciaWear **Developer:** Pablo Suárez Varela **Role:** Full-stack developer (solo project, TFG DAM 2024-26)

GaliciaWear is a cross-platform sustainable-fashion marketplace connecting independent Galician designers with environmentally-conscious consumers. The platform provides certified sustainability filters (GOTS, OEKO-TEX, km 0), real-time designer chat, and eco-friendly delivery via Correos Verde.

2. Problem & Solution

Pain point: Local Galician fashion designers lack a professional digital channel. Existing marketplaces (Amazon, Zalando) prioritize global fast-fashion brands with no sustainability certification or geographic focus.

Solution: A four-platform system (REST API + React web + Android app + JavaFX admin) with sustainability-first search filters, ACID-safe inventory management, Socket.IO real-time chat/notifications, and an automated backup system.

3. Main Features

- Authentication & RBAC: JWT access tokens (15 min) + rotating opaque refresh tokens (7 days, stored hashed SHA-256). Three roles: CLIENTE, DISEÑADOR, ADMIN.
- CRUD across platforms: 66 REST endpoints covering 13 modules, all documented in OpenAPI 3.0.3 (Swagger UI at /api/docs).
- Real-time notifications: Socket.IO personal rooms (usuario:sub) for in-app notifications; Firebase Cloud Messaging for push.
- Automated backup & XML/JSON export: daily backup.sh (mysqldump + mongodump, 30-day retention); admin endpoints for full catalog export/import.
- Sustainability engine: product filter by certificate (GOTS/OEKO-TEX/GRS), km of origin, and material type with Prisma relation filters.

4. Technologies Used

Layer	Technology
Languages	TypeScript 5 (backend + web) · Java 17 (Android + JavaFX)
Backend	Node.js 20 · Express 4 · Prisma ORM 6 · Zod · Socket.IO 4
Databases	MySQL 8 (relational, 17 models) · MongoDB 7 (NoSQL, 6 collections with TTL)
Web frontend	React 18 · Vite 5 · TailwindCSS 3 · TypeScript
Android	Java 17 · MVVM · Hilt · Room · Retrofit · Socket.IO Java client
Desktop	JavaFX 21 · Maven · jpackage (.deb / .ApplImage) · OkHttp · TestFX
Infra/CI	Docker + Docker Compose · GitHub Actions (ubuntu-latest)

Layer	Technology
Testing	Jest + Supertest · JUnit 5 + Mockito + Espresso · TestFX (157 total)

5. User Interface (UI/UX)

UI design follows evidence-based psychological principles applied consistently across all three frontends (same color palette: atlantic/galego/sand; same typographic hierarchy: Syne brand font + Inter UI):

- **Fitts's Law:** BottomNavigationBar in thumb zone; full-width CTA buttons in checkout.
- **Hick's Law:** 3-slide onboarding; max 5 bottom tabs; chips for variant selection instead of nested dropdowns.
- **Cognitive load reduction:** Sustainability badge chips instead of text lists; order summary before payment confirmation.
- **Error prevention:** Submit button disabled until form validation passes; AlertDialog for destructive actions (logout, order cancel).
- **Accessibility:** textoAlternativo field in product images; WCAG AA contrast ratio in corporate palette.

6. Future Improvements

- AI-assisted features: personalized recommendations using customer eco-profile (MongoDB Recommendations collection already modeled).
- OAuth 2.0 with Google: scaffolded in .env.example; reduces registration friction.
- Full-text search: MySQL GIN indexes + Typesense for fast catalog search.
- Enhanced security: OAuth2 / 2FA, audit log export, rate-limit per user.
- iOS app: React Native or Swift client consuming the same REST API.
- Cross-border expansion: i18n for Galician (gl), Spanish (es), English (en).

7. Conclusion

GaliciaWear demonstrates production-grade software engineering applied to a real social need: the digitalization of Galicia's sustainable fashion ecosystem. The project integrates all DAM curriculum modules (relational + NoSQL databases, OOP, cross-platform development, REST APIs, real-time communications, security, testing) into a cohesive, deployable product. Technical competencies reinforced include ACID transaction design, WebSocket architecture, mobile MVVM with offline-first caching, and CI/CD pipeline design. The codebase is ready for production deployment and provides a solid foundation for future growth.